Prova Técnica — Full Stack

Nome: _____ Data: ____/____/_____

Instruções

A prova tem 24 questões em 4 partes e um desafio prático separado. Todas partem de situações reais do nosso dia a dia, adaptadas para um sistema fictício.

O sistema "Órbita". Plataforma interna de uma operadora de telecom, usada por milhares de operadores de atendimento e vendas. Módulos citados na prova:

- **Demandas:** filas de chamados que um operador reserva, trata e encerra.
- **Cobertura:** cerca de 20 milhões de endereços vindos de duas fornecedoras de rede, com deduplicação, bloqueios por UF, município, bairro ou CEP, carga diária e exportação de bases com centenas de milhões de linhas.
- **Atendimento guiado:** tela que conduz o operador passo a passo e, no fim, abre uma demanda.
- **Relatórios e arquivos:** geração assíncrona e download de arquivos de centenas de MB.

Stack. Node.js 20 + TypeScript; API em Fastify 5 com validação Zod e JWT com escopos; MySQL 8 sem ORM (`mysql2` com SQL escrito à mão); Redis para cache, travas e pub/sub; MongoDB em alguns serviços; workers Node para jobs; frontend Angular 21 + Ionic 8 (componentes standalone, signals, OnPush, formulários reativos) que roda como PWA e app mobile; processos gerenciados por PM2; deploy por GitLab CI.

Regras.

1. Responda em português. Código pode ser TypeScript, SQL ou pseudocódigo claro; não precisa compilar.
2. Explique o porquê. Uma resposta curta e certa, com a razão, vale mais que uma longa.
3. "Não sei, mas investigaria assim" é uma resposta válida e pontua. Inventar API ou comportamento perde ponto.
4. Pode consultar a documentação oficial (MDN, Angular, Fastify, Zod, MySQL, Node). Não pode usar IA nem falar com outras pessoas.
5. Tempo sugerido: 2 h 30 para as Partes A a D. O tempo indicado em cada questão é referência, não limite.

Parte	Tema	Questões	Pontos (0 a 3 por questão)
A	Backend	1 a 7	21
B	Banco de dados	8 a 13	18
C	Frontend	14 a 19	18
D	Arquitetura e operação	20 a 24	15
Total		24	72

Parte A — Backend

1. Rota com Zod e escopos (≈ 8 min)

Todas as rotas da API seguem este formato:

```
fastify.withTypeProvider<ZodTypeProvider>().post(
  '/attendance/:uuid/finish',
  {
    schema: {
      params: AttendanceParamsSchema,
      body: FinishBodySchema,
      response: { 200: AttendanceSchema, 404: ErrorSchema, 409: ErrorSchema },
    },
    preHandler: haveScope(['attendance', 'attendance:supervise']),
  },
  async (req, res) => { /* ... */ },
);
```

a) Escreva o `FinishBodySchema` em Zod: `outcome` é `'success'` ou `'failure'`; quando for `'failure'`, `failureReasonKey` (texto) é obrigatório; `note` é opcional, sem espaços nas pontas e com no máximo 500 caracteres.

b) Além de documentar, o que o `response` faz em tempo de execução? O que acontece se o objeto de domínio devolvido pelo handler tiver um campo `ownerPasswordHash` que não está no `AttendanceSchema`?

c) A regra é: o operador só encerra o próprio atendimento; a supervisão encerra o de qualquer um. O `haveScope` com dois escopos resolve isso sozinho? Onde você colocaria essa regra?

2. Duas pessoas reservando a mesma demanda (≈ 8 min)

```
export async function reserveDemand(conn: Conn, userId: number, demandId: number) {
  const [demand] = await conn.query(
    'SELECT id, worker_id FROM demand WHERE id = ?', [demandId]);
  if (demand.worker_id) throw { msg: 'Demanda já reservada' };

  await conn.execute(
    'UPDATE demand SET worker_id = ?, status = 3 WHERE id = ?', [userId, demandId]);
  await conn.execute(
    'INSERT INTO demand_trace (demand_id, user_id, note) VALUES (?, ?, ?)',
    [demandId, userId, 'Reservado para tratamento']);
}
```

Em produção, duas pessoas clicaram em "Reservar" quase juntas. As duas viram sucesso, a demanda ficou com a segunda e o histórico mostra duas reservas.

a) Explique a causa.

b) Corrija sem segurar a linha travada enquanto o usuário pensa. O perdedor deve receber uma mensagem dizendo quem ficou com a demanda.

c) O mesmo operador, com a demanda aberta em duas abas, clica nas duas. O que deveria acontecer na segunda aba?

3. Download que derruba o processo (≈ 7 min)

```
fastify.get('/file/:id', async (req) => {
  const file = await getFile(req.params.id);
  const data = fs.readFileSync(file.path).toString('base64');
  return { name: file.name, type: file.type, data };
});
```

Relatórios chegam a 300 MB. Três processos da API morreram ao entregar um ZIP de 142 MB, sem nem registrar a requisição no log.

a) Por que um arquivo de 142 MB derruba um processo que tem mais de 1 GB livre?

b) Reescreva a rota.

c) O app mobile e o PWA antigos esperam `data` em base64 e continuam em uso por dias depois do deploy. Como trocar o formato sem quebrar quem ainda não atualizou?

4. Login por ticket de uso único (≈ 8 min)

O sistema do call center (outro time, outra rede) precisa abrir o Órbita já autenticado como o operador, sem ele digitar senha. Fluxo proposto:

1. O call center chama `POST /sso/ticket` (servidor a servidor, com credencial própria) informando o login do operador e recebe um ticket.
2. Abre o navegador em `https://orbita.exemplo/sign/sso#ticket=<ticket>`.
3. O frontend troca o ticket por um JWT.

Requisitos: o ticket vale 120 segundos, só pode ser usado uma vez e perfis administrativos nunca entram por ticket.

a) Como você gera, guarda e consome o ticket? Onde ele fica e com qual chave?

b) Como garantir que dois pedidos simultâneos com o mesmo ticket não gerem duas sessões?

c) Por que o ticket vai depois do `#` e não como `?ticket=`?

5. Integração externa instável (≈ 6 min)

```
async function geocode(cep: string) {
  const { data } = await axios.get(`https://geo.fornecedor.com/cep/${cep}`);
  return { lat: data.lat, lng: data.lng };
}
```

Essa função roda dentro da consulta de viabilidade. Às vezes o fornecedor demora 60 s, às vezes responde `204` sem corpo, e em produção a saída para a internet passa por um proxy corporativo. Os operadores dizem que a tela "congela".

Liste os problemas e reescreva a função.

6. Dado sensível e log (≈ 5 min)

A API grava toda requisição e toda resposta numa coleção de auditoria. A rota de cliente devolve CPF e telefone mascarados, e o valor completo só para quem tem o escopo `attendance:unmask`.

a) A máscara pode ficar só no frontend? Por quê?

b) Que problema existe em gravar o corpo da resposta dessa rota e da rota de login? Como você resolveria de forma que ninguém esqueça na próxima rota?

c) Em produção, a recusa de login devolvia um campo `debug` com dados internos da conta. Que regra geral isso quebra e como impedir que volte a acontecer?

7. Teste da correção (≈ 8 min)

Usando a sua correção da questão 2:

a) Liste os casos de teste que você escreveria.

b) Escreva com Vitest o caso "perdeu a corrida para outra pessoa". A conexão pode ser um objeto falso (`query`, `execute`, `begin`, `commit`, `rollback`). Verifique o que deve e o que não deve ter sido chamado.

Parte B — Banco de dados (MySQL 8)

8. Recálculo que não termina (≈ 8 min)

```
-- address: 20 milhões de linhas
-- block:   110 mil linhas, KEY ix_level_value (level, value)
--          level 1 = UF, 2 = município, 3 = CEP
UPDATE address a
SET a.blocked = EXISTS (
  SELECT 1 FROM block b
  WHERE b.active = 1
    AND (   (b.level = 1 AND b.value = a.state)
         OR (b.level = 2 AND b.value = a.city_key)
         OR (b.level = 3 AND b.value = a.zipcode))
)
WHERE a.state = 'AC';
```

Com a tabela `block` vazia, o lote do Acre (94 mil endereços) rodava em segundos. Depois da carga de 110 mil bloqueios, passou de 80 minutos sem terminar.

a) Por que o índice `ix_level_value` não ajuda aqui?

b) Reescreva para usar o índice.

c) O que você procuraria no `EXPLAIN` antes e depois para provar a melhora?

9. JSON que quebra só em produção (≈ 5 min)

```
SELECT a.id,
       CAST(CONCAT('[',
         GROUP_CONCAT(JSON_OBJECT('reason', b.reason, 'priority', b.priority)
                      ORDER BY b.priority DESC),
       ']') AS JSON) AS blocks
FROM address a
JOIN address_block ab ON ab.address_id = a.id
JOIN block b ON b.id = ab.block_id
GROUP BY a.id;
```

Em homologação funciona. Em produção, alguns endereços dão erro de JSON inválido, sempre os que têm muitos bloqueios. Além disso, usuários reclamam que a ordem dos motivos na tela muda de um dia para o outro.

Explique os dois problemas e corrija.

10. Vazio que não é vazio (≈ 5 min)

a) O grid tem o filtro "(Em Branco)" na coluna `complement` (VARCHAR, aceita NULL). O backend gera `WHERE complement IS NULL` e o filtro traz menos linhas do que o esperado. O que está acontecendo e como corrigir?

b) Esta consulta passou a devolver zero linhas depois que alguém inseriu um registro em `disabled_supplier`. Por quê?

```
SELECT * FROM address
WHERE supplier NOT IN (SELECT supplier FROM disabled_supplier);
```

11. Deduplicação de endereços (≈ 8 min)

As duas fornecedoras mandam o mesmo imóvel com o número escrito de formas diferentes: `''`, `NULL`, `'SN'`, `'S/N'`, `'s/n'`, `'0'`, `'11'` e `'11_'`. A exportação já grava como `SN` todo número vazio, zero ou sem nenhum dígito, e a deduplicação não seguia essa regra. Resultado: o mesmo imóvel aparecia duas vezes no arquivo exportado.

a) Escreva em TypeScript `numberKey(raw: string | null): string`, a chave de número usada na deduplicação.

b) Com a regra nova no ar, já existem 500 grupos de endereços duplicados no banco. Cada endereço tem filhos em outras tabelas (vínculos com a origem, bloqueios, complementos). Descreva como fundir os grupos com segurança.

c) Por que não simplesmente sobrescrever o número gravado na tabela pela versão normalizada?

12. Filtro e ordenação vindos do grid (≈ 7 min)

```
const allowed = ['state', 'city', 'zipcode', 'status'];
let where = '1=1';
for (const [col, f] of Object.entries(req.body.filterModel)) {
  if (!allowed.includes(col)) continue;
  where += ` AND ${col} IN ('${f.values.join("','")}')`;
}
const { colId, sort } = req.body.sortModel[0];
const sql = `SELECT * FROM address WHERE ${where}
             ORDER BY ${colId} ${sort}
             LIMIT ${req.body.startRow}, ${req.body.endRow - req.body.startRow}`;
```

Aponte todos os problemas (segurança, comportamento e desempenho) e reescreva. Um deles não é de segurança: o que acontece com o usuário quando ele filtra por uma coluna que não está em `allowed`?

13. Mudança em tabela grande (≈ 6 min)

Você precisa criar a coluna `technology` em `address` (20 milhões de linhas, uso 24 h por dia) e preenchê-la a partir de outra tabela.

a) Como aplicar o DDL com o menor impacto possível?

b) Como fazer o preenchimento sem travar a tabela nem atrasar a réplica?

c) Num outro dia, um `UPDATE` simples em `report` falha com uma mensagem genérica, sem relação aparente com o seu código. O que você confere no banco antes de mexer no código?

Parte C — Frontend (Angular 21 + Ionic)

Convenções do projeto: componentes standalone, `ChangeDetectionStrategy.OnPush`, `input()` / `output()`, `signal()` / `computed()`, `inject()`, controle de fluxo nativo (`@if`, `@for`) e formulários reativos.

14. Componente de dado mascarado (≈ 8 min)

Escreva o componente `app-masked-value`:

- recebe `field: { masked: string; raw?: string } | null` e `empty` (texto padrão `'—'`);
- sem `field`, mostra `empty`;
- mostra `masked` e, só quando `raw` veio preenchido, um botão com ícone de olho que alterna entre `masked` e `raw`;
- o botão precisa ser acessível para leitor de tela e teclado.

Pode escrever template e classe no mesmo arquivo.

15. O valor que só chega no blur (≈ 6 min)

Um componente próprio `app-text-field` envolve o `ion-textarea` e implementa `ControlValueAccessor`, repassando o valor ao formulário com `(ionChange)="onChange($event.detail.value)"`. No encerramento do atendimento, a observação é obrigatória quando o motivo é "Outro", e o botão "Concluir" fica desabilitado enquanto o formulário for inválido.

a) Os operadores dizem que digitam a observação e o botão continua desabilitado até clicarem fora do campo. Por quê? Como corrigir, e qual o alcance dessa correção no resto do sistema?

b) Em outra tela, o operador digita o CPF e aperta Enter. A busca sai com o valor da busca anterior, mas só quando o campo já tinha algo antes. O `ion-input` repassa o valor num `setTimeout(0)`. Explique a sequência de eventos e proponha uma correção.

16. OnPush que não atualiza (≈ 5 min)

```
@Component({
  selector: 'app-block-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @for (b of blocks; track $index) { <app-block-row [block]="b" /> }
    <span>{{ blocks.length }} bloqueios</span>`,
})
export class BlockListComponent {
  @Input() blocks: Block[] = [];
}

// no componente pai
addBlock(b: Block) { this.blocks.push(b); }
```

a) O bloqueio novo não aparece na lista. Por quê?

b) Reescreva pai e filho com signals (`signal` no pai, `input()` e `computed()` no filho).

c) O que muda trocar `track $index` por `track b.id` quando o usuário remove um item do meio de uma lista de 2 mil linhas?

17. Sair da tela com atendimento aberto (≈ 7 min)

Ao sair do cockpit com o atendimento em andamento, o sistema deve perguntar: "Encerrar atendimento", "Sair e cancelar atendimento" ou "Continuar no atendimento". Existe `ask(message: string, labels: string[]): Promise<number | null>`, que devolve o índice do botão clicado ou `null` se a pessoa fechar o diálogo.

a) Escreva o `CanDeactivateFn<CockpitPage>`. "Sair e cancelar" chama `page.store.abandon()` antes de sair; "Encerrar" abre o formulário de encerramento e fica na tela.

b) Dentro do cockpit há links para a demanda vinculada. O que acontece se forem `routerLink` comuns e o que você faria?

c) Recarregar a página ou fechar a aba não passa pelo guard. Como o sistema deve se comportar nesses casos, no frontend e no backend?

18. Busca enquanto digita (≈ 6 min)

a) O campo de logradouro consulta a API enquanto o usuário digita. Requisitos: esperar 400 ms de pausa, só consultar com 3 caracteres ou mais, cancelar a requisição anterior quando chega uma nova, mostrar "carregando" e, se uma consulta falhar, mostrar o erro sem parar de buscar nas próximas digitações. Escreva com RxJS ou com `rxResource` / `resource`.

b) Para acompanhar uma exportação, um colega escreveu `api.call('/export/status', 'get', {})` a cada 5 s, e a requisição nunca aparece na aba Network. O que aconteceu?

19. Conteúdo em Markdown escrito pela operação (≈ 4 min)

A equipe de operação escreve as falas do atendimento num painel de administração, em Markdown, e o cockpit as exibe formatadas. Um colega fez:

```
<div [innerHTML]="sanitizer.bypassSecurityTrustHtml(marked.parse(text))"></div>
```

porque "o Angular estava tirando o estilo". Qual o risco, quem poderia explorá-lo e qual a forma certa?

Parte D — Arquitetura, operação e depuração

20. A fila que parou por 4 horas (≈ 7 min)

Um worker executa sincronizações em fila, uma de cada vez. Cada uma lê uma consulta da origem (SQL Server ou MySQL) por stream, grava um arquivo NDJSON e envia para um data warehouse.

a) Encontre o defeito:

```
const out = fs.createWriteStream(path);
for await (const row of source) {          // source: Readable da origem
  if (!out.write(JSON.stringify(row) + '\n')) {
    await once(source, 'drain');
  }
}
```

b) Corrigido isso, um stream do SQL Server parou de entregar linhas, sem erro, e todas as sincronizações atrás dele ficaram paradas por 4 horas. Como você colocaria um prazo na leitura?

c) Para a origem MySQL (`mysql2` com stream e conexão de pool), um colega sugere chamar `stream.destroy()` quando o prazo estourar. Que risco isso traz para o pool? Que alternativa existe do lado do servidor, e que cuidado ela exige antes de devolver a conexão ao pool?

21. Reinício que mata o job (≈ 6 min)

O PM2 recarrega todos os processos todo dia às 03:00 e a cada deploy. A exportação das bases leva cerca de 40 minutos e gera arquivos de vários GB. Certo dia, a exportação foi morta no meio: ficou um arquivo pela metade no destino e o status "executando" na tela para sempre.

a) O que você muda no job para que um reinício não deixe estado inconsistente (arquivo, status e trava)?

b) O que muda no processo de deploy?

22. Backend e frontend publicados em tempos diferentes (≈ 6 min)

O CI publica o backend cerca de 6 minutos antes do frontend. Além disso, o PWA (com service worker) e o app mobile (atualização OTA) levam dias para chegar a todos os usuários. Cada requisição envia a versão do app num header.

Você precisa renomear o campo `hp` para `hpTotal` e trocar `status` de número (`1`, `2`, `3`) para texto (`'active'`, `'blocked'`, `'closed'`) numa resposta muito usada. Descreva o plano, passo a passo, sem quebrar ninguém.

23. Trava durante a carga (≈ 6 min)

A carga das bases leva até 2 horas e roda num worker separado. Enquanto ela roda, ninguém pode editar bloqueios nem parametrizações. A API roda em 4 instâncias (cluster do PM2).

a) Desenhe a trava: onde fica, como é adquirida, renovada e liberada, e o que acontece se o worker morrer no meio.

b) Por que uma variável em memória não serve? E uma linha `locked = 1` numa tabela do MySQL, sem mais nada?

c) Como a API e a tela se comportam quando alguém tenta editar com a trava ativa?

24. Incidente: e-mails repetidos no ambiente de QA (≈ 7 min)

```
const isProd = env.NODE_ENV === 'production';

for (const email of await Email.find({ sent: null })) {
  const lock = await redis.set(`${prefix}:email:${email.uuid}`, '1', 'EX', 86400, 'NX');
  if (!lock) continue;

  await transporter.sendMail(toMessage(email));
  if (isProd) {
    await Email.updateOne({ uuid: email.uuid }, { $set: { sent: new Date() } });
  }
}
```

No ambiente de QA, os usuários recebem os mesmos e-mails de novo a cada 24 horas, e todo worker novo (que sobe com outro `prefix` de Redis) reenvia a fila inteira.

a) Qual a causa?

b) Corrija. Considere também o processo morrer entre o `sendMail` e o `updateOne`: o que acontece, e qual garantia você prefere (no máximo uma vez ou pelo menos uma vez)?

c) O `isProd` existia porque o `.env` de desenvolvimento local aponta para o MongoDB de produção, e o desenvolvedor não podia dar baixa nos e-mails reais. O que você acha dessa configuração e o que proporia?

Parte E — Desafio prático

Construa uma versão mínima da fila de Demandas, com backend e frontend, em até 4 horas de trabalho. Prazo de entrega: 3 dias corridos a partir do envio. Pode usar IA, desde que declare no README onde e como.

Banco (fornecido pelo time em `docker-compose.yml`, com 5 mil demandas de exemplo):

```
CREATE TABLE app_user (
  id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);
CREATE TABLE demand (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  queue VARCHAR(40) NOT NULL,
  status TINYINT NOT NULL DEFAULT 1,      -- 1 aberta, 2 reservada, 3 encerrada
  worker_id INT NULL,
  outcome ENUM('success','failure') NULL,
  failure_reason VARCHAR(40) NULL,
  note VARCHAR(500) NULL,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  KEY ix_queue_status (queue, status)
);
CREATE TABLE demand_trace (
  id INT AUTO_INCREMENT PRIMARY KEY,
  demand_id INT NOT NULL,
  user_id INT NOT NULL,
  action VARCHAR(20) NOT NULL,
  note VARCHAR(500) NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Backend (Node + TypeScript, Fastify, Zod, `mysql2` sem ORM). Para simplificar a autenticação, o usuário vem no header `x-user-id` e os escopos em `x-scopes`, separados por vírgula.

1. `GET /demands`: paginação no servidor, filtro por `queue` e `status`, ordenação por `id` ou `updated_at`, devolvendo também o total. Campo de filtro ou ordenação desconhecido devolve `400`.
2. `POST /demands/:id/reserve`: segura contra duas pessoas ao mesmo tempo. Quem perde recebe `409` com o nome de quem ficou. O mesmo usuário reservando de novo recebe sucesso.
3. `POST /demands/:id/finish`: corpo validado com Zod (`outcome`; `failureReason` obrigatório no insucesso; `note` opcional, até 500 caracteres). Só o dono da reserva ou quem tem o escopo `demands:supervise` pode encerrar.
4. Toda mudança grava uma linha em `demand_trace`, na mesma transação.

Frontend (Angular standalone, OnPush, signals, formulários reativos; Ionic é opcional).

1. Lista paginada com filtro de fila e status e botão "Reservar". Um `409` mostra a mensagem e atualiza a linha, sem recarregar a página.
2. Formulário de encerramento com a validação condicional do motivo e o botão desabilitado enquanto inválido.

Testes. No mínimo, um teste que prove que duas reservas simultâneas da mesma demanda resultam em um sucesso e um `409`.

Entrega. Repositório Git com histórico de commits e um README com: como rodar, decisões tomadas, o que faria com mais tempo e uso de IA.

Na defesa oral, você vai alterar o próprio código ao vivo, então chegue conhecendo bem o que entregou.